

The OpenPGP standard defines GPG (Gnu Privacy Guard) as an open source implementation of OpenPGP to allow users to encrypt files using hybrid cryptography. This uses both symmetric and asymmetric encryption, i.e., data is encrypted with a random generated symmetric key, and the symmetric key is also encrypted using the public key of the recipient. GPG also provides the ability to digitally sign files, which confirm the authenticity of the user. It has historically been used as the standard and most widely used application for encrypted email and signing of software distributions.

For teams that do not want to deal with the complexities of GPG, Age (Actual Good Encryption) provides a simple alternative with the use of X25519 keys. No configuration files are needed, and its simple method to encrypt files makes it very effective for backing up and automating processes.

Database and application-level encryption

Database systems must receive special consideration since they hold the largest and most sensitive records. The pgcrypto extension of PostgreSQL allows for column-level encryption, meaning that national identity numbers and payment information are stored as encrypted data (ciphertext) when at rest (i.e., when not being processed), thereby protecting these fields from anyone who accesses a raw dump of the database files. Both MariaDB and MySQL support tablespace activity records and are transparent to both the data-at-rest encrypted version and the log file. Percona provides open source encryption related applications for MongoDB. The best design uses application layer encryption to encrypt sensitive fields before submission to the database, meaning that neither the database administrator or the backup operator has ever accessed that data as plain text.

Key management: The critical foundation

The process of encryption turns the question of how to protect data into that of how to protect keys. The problems associated with the management of keys are the main reason for the failure of many real-world encrypted applications. A key that is hard coded into a program, checked into a Git repository, or found next to the data it is supposed to secure only creates an illusion of security. To ensure there is actual protection, sound practice is dependent on several principles including but not limited to:

- **Separate keys from data:** Store encryption keys in a dedicated system, never on the same disk or in the same database as the ciphertext they unlock.
- **Centralise secrets:** HashiCorp Vault and the fully open source fork OpenBao provide encrypted storage for keys and credentials, fine-grained access policies, complete audit logs, and an encryption-as-a-service API that lets applications encrypt data without ever holding the key material themselves.
- **Rotate and version keys:** Regular rotation limits the blast radius of a leaked key. Envelope encryption wrapping many data keys with a single master key makes rotation practical at scale.
- **Plan for recovery:** Lost keys mean permanently lost data. Maintain securely stored, access-controlled recovery keys and test the restoration procedure, not just the backup.
- **Enforce least privilege:** Humans and services should access only the keys they require, for the shortest time necessary, with every access recorded.

Challenges and practical considerations

There are several obstacles to deploying encryption and planning ahead for these will surely help to build resilient systems. Although performance is the

primary concern, in reality it is primarily a legacy issue. AES encryption is almost costless on hardware that has AES-NI capabilities, which will be just about every CPU built in the last several years. Also, TLS 1.3 has resulted in far less connection setup delays compared to previous versions of TLS.

Much more significant than performance-related concerns are those associated with operations. Expired certificates are still one of the leading causes for downtime across all platforms. NCertbot and ACME should be viewed as the preferred option for automating the deployment of encryption keys. The issue of unattended reboots due to full-disk encryption on servers can be solved by using tools for network-bound disk encryption (e.g., Clevis and Tang) that will automatically release an encryption key when a server connects to the trusted network.

Encryption can create a conflict with different types of functions; for example, an encrypted database column cannot be indexed, searched, or have regular monitoring on the encrypted traffic that was sent through a legacy network. So, how do we mitigate these negative effects on our overall security? One way is to only encrypt specific columns that contain sensitive information and to do most of the inspection via the endpoints rather than traditional methods like monitoring the network through a monitoring program.

Open source software requires constant monitoring and updates to keep everything secure; this was very clearly demonstrated by the Heartbleed vulnerability in 2014, which showed that even very commonly used encryption algorithms have not been updated quickly enough since their last release to fix a major security issue. However, since these types of issues are in the public domain, they are usually fixed much faster than those that depend on a company's willingness to fix them.

■■ Continued to page...49